

SYSTEM DESIGN INTERVIEW

Design Google Meet

Group video conferencing at scale: WebRTC, SFU topology, adaptive streaming, recording

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · Topology Choice: Mesh vs MCU vs SFU
- 05** · High-Level Architecture
- 06** · Signaling Plane
- 07** · Media Plane: SFU + Simulcast
- 08** · Active Speaker Detection
- 09** · Recording (Phantom Participant)
- 10** · Live Captions & Translation
- 11** · Adaptive Streaming & Congestion Control

- 12** · Scalability & Global Distribution
- 13** · Comparison: Meet vs Zoom vs Teams
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Requirements and scope

Design a real-time group video conferencing platform like **Google Meet**, **Zoom**, or **Microsoft Teams**. The service must support multi-party audio/video calls, screen sharing, recording, and live captions, all running over the public internet at planetary scale.

Clarifying Questions

Question	Answer
Concurrent participants peak?	~100M global (post-COVID baseline)
Avg call shape?	4 participants, 30 minutes
Calls/day?	~10M
Max participants per call?	500 active video, up to 100K view-only (livestream)
Devices?	Browser (WebRTC), iOS, Android, native desktop
Recording?	Yes; cloud recording to GCS, optional captions burn-in
Captions / translation?	Real-time ASR; live translation to ~30 languages
E2E encryption?	Optional client-side E2EE for <= 200 participants

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Join / leave	Authenticated room join via meeting code or link
Audio + video	Bidirectional A/V; mute / camera toggle; pin / spotlight
Screen share	Share window or full screen as additional video track
Adaptive quality	Auto-adjust resolution/bitrate to match downlink BW
Recording	Cloud recording with composed layout; upload to GCS
Live captions	Speech-to-text streamed back via data channel
Chat + reactions	Text chat sidecar; ephemeral emoji reactions

Non-Functional Requirements

Requirement	Target
End-to-end latency	< 150 ms mouth-to-ear at p50; < 300 ms at p99
Availability	99.99% (4 nines); regional failover within 30 s
Concurrent participants	100M global peak
Calls / day	10M; 4 participants × 30 min average
Quality	VP9 / AV1 video; Opus audio; FEC + RED for loss
Loss tolerance	Smooth at 5% packet loss; degraded at 20%

03

Capacity Estimation

Math for scale

Traffic Estimation

- Concurrent participants peak: **100M** globally
- Average call: **4 participants × 30 min**
- Calls/day: **~10M** (each contributes 4 participant-sessions)
- Participant-sessions/day: $10M \times 4 = \mathbf{40M}$
- Participant-minutes/day: $40M \times 30 \text{ min} = \mathbf{1.2B \text{ participant-minutes}}$
- Steady-state concurrent (avg): $1.2B / 1440 \text{ min} \approx \mathbf{833K}$; peak amplifies $\sim 120\times$ to 100M

Bandwidth per Participant

- **Uplink**: 1 video stream out at $\sim 1.5 \text{ Mbps}$ (720p, simulcast L3) + audio at 40 kbps Opus
- **Downlink (1-on-1)**: 1 stream in $\approx 1.5 \text{ Mbps}$
- **Downlink (4-person call)**: 3 streams $\times \sim 0.5 \text{ Mbps}$ (mid layer) $\approx 1.5 \text{ Mbps}$
- **Downlink (25-tile gallery)**: 24 streams $\times \sim 150 \text{ kbps}$ (low layer) $\approx 3.6 \text{ Mbps}$
- **Downlink (100-person webinar, all visible)**: up to 25 Mbps; SFU only forwards **top-N by speaker** ($N \approx 9$)

Aggregate SFU Capacity

- Per participant ingress (1 stream up): $\sim 1.5 \text{ Mbps}$ audio+video
- Per participant egress (avg 5 streams down): $\sim 3 \text{ Mbps}$
- Per SFU box (32-core, 100 Gbps NIC): $\sim 5K$ participants peak
- 100M peak / 5K per box = **20,000 SFU boxes** globally
- Distribute across **~30 regions**; each region $\approx 600\text{--}800$ SFUs

Storage (Recordings)

- Recording bitrate: $\sim 2 \text{ Mbps}$ composed (1080p layout + audio)
- Per recorded minute: $2 \text{ Mbps} \times 60 / 8 = \mathbf{15 \text{ MB}}$
- If 5% of calls record (500K/day) avg 30 min: $500K \times 30 \times 15 \text{ MB} = \mathbf{225 \text{ TB/day}}$
- 90-day retention default: **~20 PB** in GCS Standard + Coldline

KEY

Key Numbers

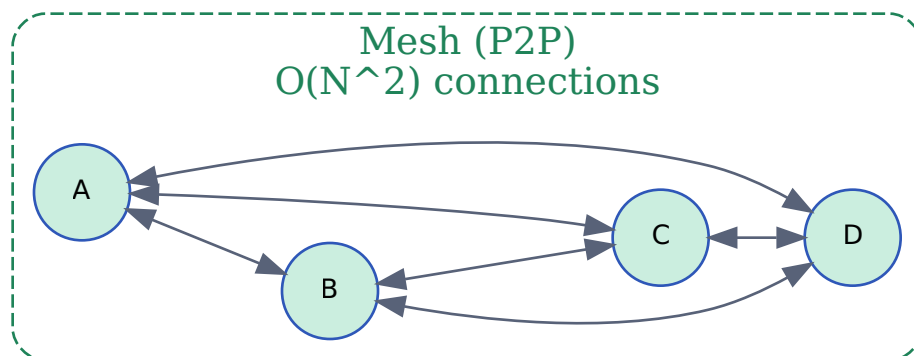
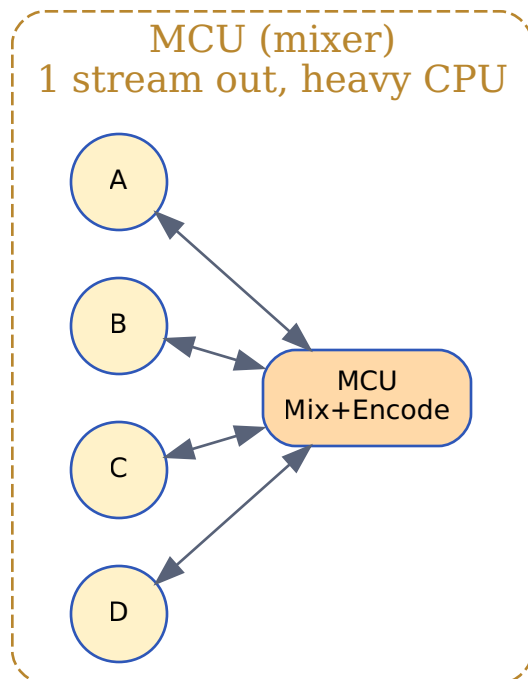
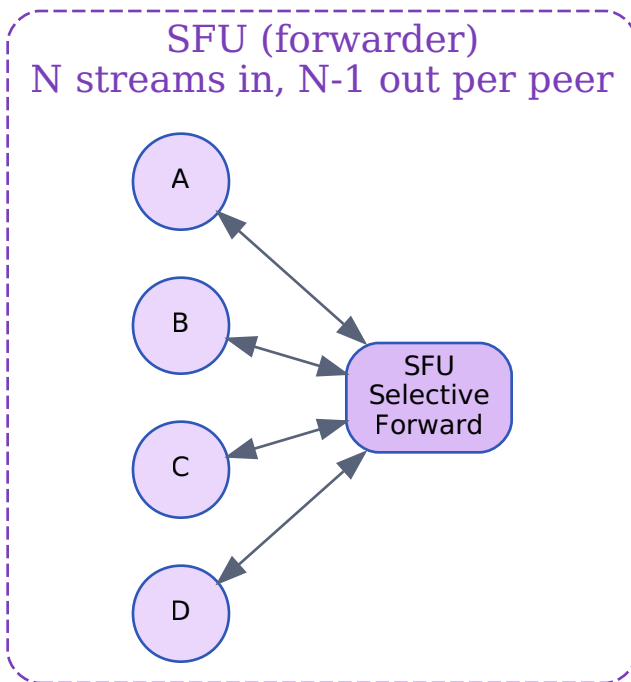
Peak concurrent: **100M** · Calls/day: **10M** · SFU fleet: **~20K boxes** across 30 regions · Recordings: **~225 TB/day** · Mouth-to-ear p50 < **150 ms**.

04

Topology Choice: Mesh vs MCU vs SFU

Why Google chose SFU + simulcast

The single largest architectural decision is **how media flows** between participants. There are three classical topologies; all real products at scale converge on SFU.



Mesh (P2P, full graph) vs MCU (server mixes one stream) vs SFU (server forwards selectively).

Comparison Table

Aspect	Mesh (P2P)	MCU	SFU
Connections per peer	N-1	1	1
Server CPU	None	Very high (decode + mix + encode)	Low (forward only, no transcode)
Server bandwidth	0	N in + N out	N in + N×(N-1) out
Best for	< 4 participants	Legacy / phone bridge	> 4 participants (general case)
Per-receiver layout	Client	Server-fixed	Client-chosen (flexible)
Cost at 100 ppl	Infeasible (uplink saturation)	\$\$\$ GPU encoders	\$ commodity NIC
End-to-end encryption	Native	Breaks E2EE (server decrypts)	Possible via Insertable Streams

Why SFU + Simulcast Wins

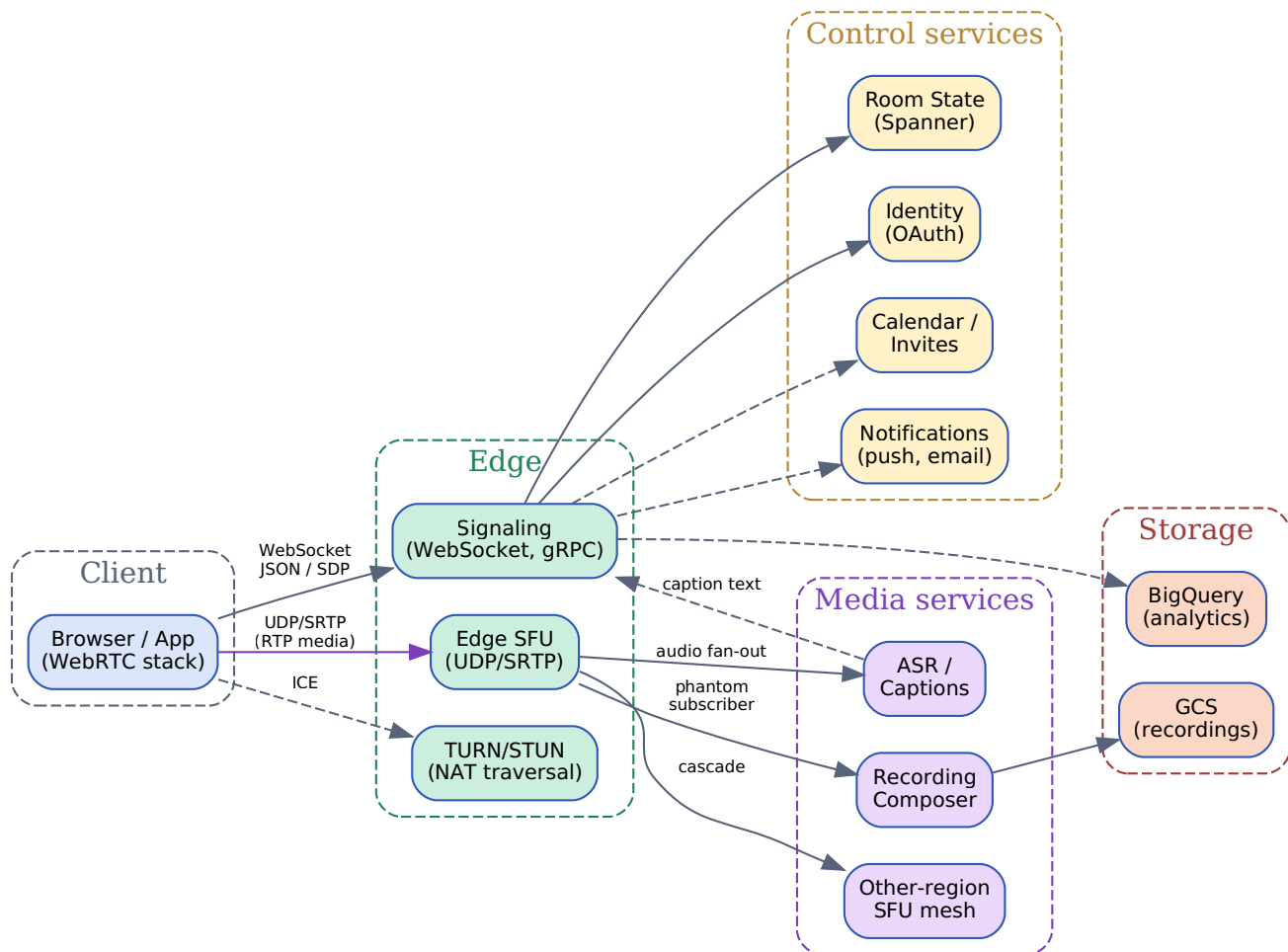
- **Linear uplink:** sender uploads exactly 1 stream regardless of room size
- **No transcode:** SFU is mostly a packet router; CPU stays cheap
- **Per-receiver adaptation:** SFU forwards a different simulcast layer to each subscriber based on their downlink
- **E2EE-compatible:** SFU does not need to decrypt payload; it only inspects RTP headers
- **Mesh fallback:** for 2-person calls, Meet uses pure P2P to bypass the SFU when ICE permits (lowest latency, no server cost)

05

High-Level Architecture

Control plane vs media plane

Meet cleanly separates the **signaling (control) plane** (WebSocket, JSON, room state, ICE) from the **media plane** (UDP/SRTP, RTP packets through the SFU). The two run on different services with different SLOs and scaling profiles.



Browser/App → Edge SFU and Signaling. Media flows over UDP/SRTP; signaling rides WebSocket; auxiliary services (recording, ASR, identity) attach to both planes.

Component Responsibilities

- **Signaling Service:** handles room join, SDP offer/answer, ICE candidate exchange, presence, mute state, chat — over WebSocket
- **Edge SFU:** terminates SRTP, forwards selected simulcast layers to each subscriber, runs active-speaker detection

- **STUN/TURN:** NAT traversal; TURN relays media when symmetric NAT blocks direct paths
- **Recording Composer:** joins as a phantom participant, composes layout via FFmpeg, uploads to GCS
- **ASR Service:** subscribes to audio streams, runs streaming speech-to-text, posts captions back via signaling data channel
- **Room State Store:** Spanner — authoritative roster, SFU assignment, recording state
- **Identity / Calendar / Notifications:** standard Google services for auth, invites, push

06

Signaling Plane

Joining a room with WebRTC

Join Sequence

1. Client opens WebSocket: `wss://meet.google.com/signal` with bearer token
2. Client sends `JOIN { meeting_code, device_caps }`
3. Signaling validates auth (Identity), checks roster cap (Spanner), and assigns an Edge SFU near the client (geo+load aware)
4. Client creates `RTCPeerConnection` and crafts an **SDP offer** (codecs, simulcast layers, ICE ufrag)
5. Signaling forwards offer to the SFU and returns the **SDP answer** with the SFU's ICE candidates
6. ICE candidates trickle: client and SFU exchange host / srflx / relay candidates over WS until a pair connects
7. DTLS handshake → SRTP keys established; RTP starts flowing
8. Client subscribes to other participants' tracks via `SUBSCRIBE { participant_id, prefer_layer }` messages

Room / Participant Schema

```
// Spanner: rooms
CREATE TABLE rooms (
  room_id          STRING(36) NOT NULL,
  meeting_code     STRING(12) NOT NULL,
  host_user_id     STRING(64) NOT NULL,
  sfu_pod_id       STRING(64),           // primary SFU assignment
  region           STRING(8),
  recording        BOOL,
  e2ee             BOOL,
  created_at       TIMESTAMP,
  ended_at         TIMESTAMP,
) PRIMARY KEY (room_id);

CREATE TABLE participants (
  room_id          STRING(36) NOT NULL,
  participant_id   STRING(36) NOT NULL,
  user_id          STRING(64),
  display_name     STRING(128),
  role             STRING(16),           // host | cohost | guest
  joined_at        TIMESTAMP,
  left_at          TIMESTAMP,
  edge_sfu         STRING(64),           // which SFU pod they hit
  audio_muted      BOOL,
```

```
video_muted    BOOL,  
) PRIMARY KEY (room_id, participant_id),  
  INTERLEAVE IN PARENT rooms ON DELETE CASCADE;
```

ICE Candidate Exchange (over WS)

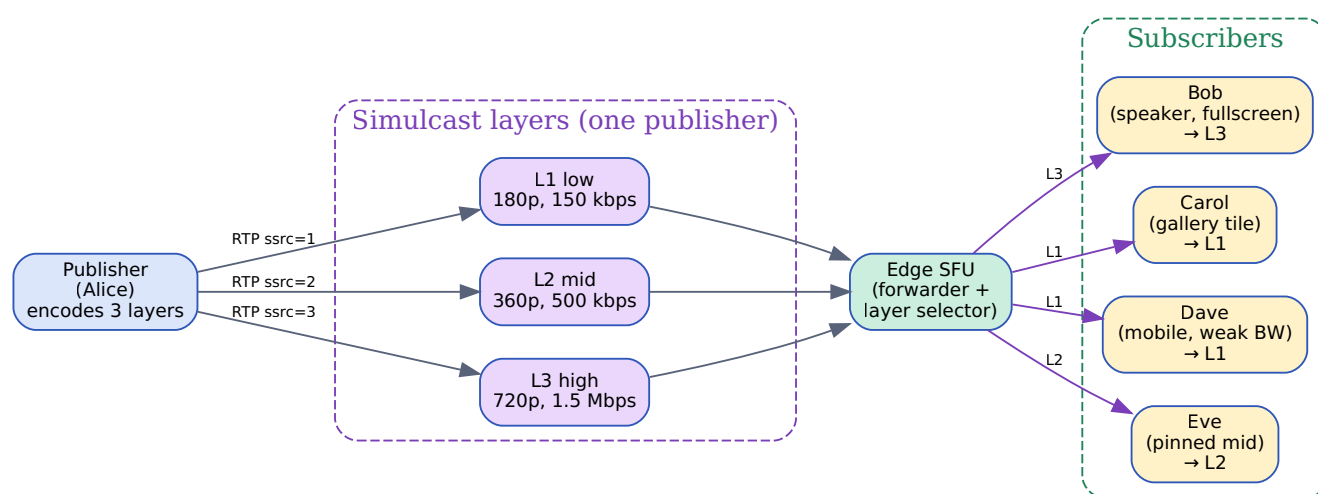
```
// client -> signaling  
{  
  "type": "ice-candidate",  
  "room_id": "7a3...",  
  "candidate": {  
    "foundation": "3",  
    "component": 1,  
    "protocol": "udp",  
    "priority": 2122260223,  
    "ip": "203.0.113.42",  
    "port": 54321,  
    "type": "srflx",           // host | srflx | relay  
    "raddr": "10.0.0.5",  
    "rport": 54321  
  }  
}  
}  
  
// SFU replies with its own candidates the same way until a pair connects.
```

07

Media Plane: SFU + Simulcast

How frames flow through the wire

Each publisher uploads **three encoded layers** of the same video (simulcast). The SFU keeps all three in memory and, for each subscriber, forwards exactly one — the one that fits the subscriber's downlink, viewport, and CPU budget.



Publisher emits L1/L2/L3 simulcast layers. SFU subscribes each receiver to one layer based on bandwidth + viewport + active-speaker rank.

Why Simulcast Beats Server-Side Transcoding

- **Cost:** client-side encoder is free; server transcoding needs GPUs
- **Latency:** SFU just routes packets — adds ~5 ms vs ~150 ms for transcode
- **E2EE-friendly:** SFU sees only encrypted payload + RTP headers; cannot transcode anyway
- **Independent rate control:** client encoder reacts to its own uplink congestion (TWCC) without SFU coordination

SVC vs Simulcast

Feature	Simulcast (VP8/VP9/H.264)	SVC (VP9-SVC, AV1-SVC)
Layers	3 separate encodes	1 encode, multiple temporal/spatial layers in 1 stream
Uplink BW	Sum of all layers (~2 Mbps)	Single bitstream (~1.5 Mbps)
CPU on sender	3× encode work	1× encode (efficient)

Feature	Simulcast (VP8/VP9/H.264)	SVC (VP9-SVC, AV1-SVC)
SFU forward	Pick one SSRC	Pick subset of NAL units per layer ID
Used by	Older Meet, Chrome default	Modern Meet (VP9-SVC), Zoom

Layer Selection Algorithm

```
# Per (subscriber, publisher) pair, every ~200 ms:
def select_layer(sub, pub, layers):
    # Inputs:
    #   sub.downlink_kbps : TWCC estimate from REMB / TWCC
    #   sub.viewport      : 'tile' | 'medium' | 'fullscreen'
    #   sub.cpu_load      : 0..1 from client stats
    #   pub.is_active     : active-speaker flag

    # 1) Cap by viewport (no point sending 720p to a 90px tile)
    cap = {'tile': 'L1', 'medium': 'L2', 'fullscreen': 'L3'}[sub.viewport]

    # 2) Boost active speaker by one layer if BW allows
    if pub.is_active and cap != 'L3':
        cap = next_higher(cap)

    # 3) Fit into remaining downlink budget
    budget = sub.downlink_kbps - sum(s.kbps for s in sub.current_subs)
    chosen = cap
    while layers[chosen].kbps > budget and chosen != 'L1':
        chosen = next_lower(chosen)

    # 4) CPU pressure: drop one layer if client is hot
    if sub.cpu_load > 0.85 and chosen != 'L1':
        chosen = next_lower(chosen)

    return chosen # SFU starts forwarding this SSRC
```

08

Active Speaker Detection

Who is talking and who gets the spotlight

The SFU continuously ranks publishers by speaking activity. The **top-N** are forwarded at high quality; tail participants drop to L1 or are entirely paused. This is the trick that lets a 100-person webinar fit on a phone.

Pipeline

1. Each Opus audio packet carries an **RTP header extension** with audio level (dBov, 0..127), per RFC 6464
2. SFU reads the header without decrypting the payload (E2EE-safe)
3. Per publisher, maintain a 200 ms moving-average energy and a 2 s short-term average
4. Apply hysteresis: a participant becomes 'active' only after ≥ 3 consecutive frames above threshold; loses it after ≥ 10 frames below
5. Rank publishers by smoothed energy $\rightarrow$ emit `active_speaker` event over signaling
6. Layer selector boosts top-N by one simulcast layer; tail beyond top-9 gets L1 only

VAD / Smoothing Algorithm

```
# Per publisher, called on every audio RTP packet (every 20 ms):
ALPHA_FAST = 0.3    # 200 ms EMA
ALPHA_SLOW = 0.05   # 2 s EMA
ENTER_DB   = -40    # dBov threshold to enter speaking
EXIT_DB    = -50    # dBov threshold to leave speaking
ENTER_FRAMES = 3
EXIT_FRAMES  = 10

def on_audio_packet(pub, lvl_dbov):
    pub.fast = ALPHA_FAST * lvl_dbov + (1 - ALPHA_FAST) * pub.fast
    pub.slow = ALPHA_SLOW * lvl_dbov + (1 - ALPHA_SLOW) * pub.slow

    if not pub-speaking and pub.fast > ENTER_DB:
        pub.streak_in += 1; pub.streak_out = 0
        if pub.streak_in >= ENTER_FRAMES:
            pub-speaking = True
            emit('active_speaker_on', pub.id)
    elif pub-speaking and pub.fast < EXIT_DB:
        pub.streak_out += 1; pub.streak_in = 0
        if pub.streak_out >= EXIT_FRAMES:
            pub-speaking = False
            emit('active_speaker_off', pub.id)

def rank_top_n(publishers, n=9):
```



```
# Use slow EMA for stable ranking, fast EMA as tiebreaker
return sorted(publishers,
               key=lambda p: (p.slow, p.fast),
               reverse=True)[:n]
```

INSIGHT

Why audio energy and not full ML?

The audio-level header extension is delivered without decoding or decrypting Opus, so a single SFU core can rank thousands of speakers in real time. ML-based VAD adds CPU and breaks E2EE. Energy + hysteresis gets > 95% of the benefit at < 1% of the cost.

09

Recording (Phantom Participant)

Server-side composer to GCS

Cloud recording is implemented as a **headless WebRTC client** that joins the call like any other participant. It subscribes to all publishers, composes them into a single 1080p frame using FFmpeg + libwebrtc, and writes a fragmented MP4 to GCS in 10-second chunks.

Composer Pipeline

1. Host clicks **Record** → signaling sets `recording=true` in Spanner and notifies all participants (consent banner)
2. Recording orchestrator spawns a **composer pod** in the same region as the SFU
3. Composer joins the room as a phantom participant (special role recorder); SFU forwards **all** publishers' L3 layers to it
4. Composer decodes each stream, lays them out (gallery / spotlight / sidebar+screenshare) on a 1920×1080 canvas at 30 fps
5. Mixes all audio streams (Opus → PCM → mix → AAC)
6. Encodes composite to H.264 + AAC; writes **fMP4 segments** (10 s) to GCS via resumable upload
7. On call end (or 12 h hard cap): finalizes manifest, transcodes to HLS, generates thumbnails, runs DLP scan, emails owner the share link

Layout Modes

Layout	Trigger	Composition
Active speaker	Default 1-on-1 / small calls	Active speaker fullscreen, others as PiP strip
Gallery	User toggle, ≤ 25 ppl	Equal-size grid; pads with avatar tiles for muted video
Spotlight	Host pins someone	Fixed pin fullscreen, others ignored
Screenshare	Anyone shares screen	Screen at 80%, presenter inset at 20%

Storage Layout

```
gs://meet-recordings/{tenant_id}/{year}/{month}/{day}/
  {meeting_id}/
    manifest.m3u8          # HLS top-level
    video/seg-00000.m4s    # 10 s fMP4 segments
    video/seg-00001.m4s
```

```
captions.vtt      # WebVTT timed text
chat.jsonl        # chat replay
thumbnail.jpg
metadata.json     # roster, duration, host
```

Lifecycle: Standard 30 days -> Nearline 60 days -> Coldline 1 yr -> delete.

10

Live Captions & Translation

Streaming ASR over the data channel

Architecture

- ASR service joins as a phantom **audio-only** subscriber per active speaker (top-3)
- Audio fans out from SFU at 16 kHz mono (downsampled from Opus); ~32 kbps per speaker
- Streaming model: chunked 100 ms frames into a Conformer-Transducer; emits partial + final tokens
- Captions posted back to signaling: { ts, speaker_id, text, is_final }
- Signaling broadcasts captions to participants via **WebRTC data channel** (in-order, reliable)
- Client renders bottom-third overlay; finalized lines are appended to the recording's WebVTT track

Translation Path

- Each finalized caption goes through NMT (neural machine translation) for the participant's preferred locale
- Cache by (source_lang, target_lang, text-hash) to avoid redundant translation cost
- Per-participant subscription: preferred_caption_lang = 'es-ES' in JOIN

Latency Budget

Stage	Budget
Audio packet → SFU → ASR	~30 ms
Streaming ASR (partial token)	~150 ms
NMT (when enabled)	~80 ms
Data channel back to client	~30 ms
Total caption-on-screen delay	< 300 ms partial; < 800 ms final

11

Adaptive Streaming & Congestion Control

Surviving the public internet

TWCC + GCC Loop

- **TWCC (Transport-Wide Congestion Control)**: receiver feeds back per-packet arrival times to sender
- **GCC (Google Congestion Control)**: sender estimates available bandwidth from one-way-delay gradient + loss
- Update period: ~100 ms; bandwidth estimate fed into encoder rate control and simulcast layer activation
- Probe packets: short bursts of padding to test for headroom without disrupting media

Loss Recovery

Mechanism	Use	Cost
NACK	Retransmit lost RTP packet	1 RTT extra latency; cheap if RTT < 50 ms
FEC (ULPFEC, FlexFEC)	Forward-error-correction parity	10–30% bandwidth overhead
RED for audio	Bundle prior frames with current	Fixed +30% audio BW; instant recovery
PLI / FIR	Force keyframe after burst loss	Big spike (~50 KB); use sparingly
Jitter buffer	Reorder + smooth playout	Adds 20–80 ms

Adaptation Ladder

1. Loss < 2%, BW healthy → keep all 3 simulcast layers; receivers pick freely
2. Loss 2–5% → enable FEC on audio; receivers downgrade to L2 if budget tight
3. Loss 5–10% → drop L3 layer entirely (sender stops encoding it); receivers max at L2
4. Loss > 10% or BW < 200 kbps → audio-only mode; pause video encoder
5. BW recovers → walk back up after 5 s of clean stats

TIP

Audio over Video, Always

When the link degrades, prioritize audio packets ruthlessly. A frozen video tile is acceptable; a chopped-up word is not. SFU marks audio with higher DSCP and skips video first under egress pressure.

12

Scalability & Global Distribution

Cascading SFUs across regions

Edge SFU Selection

- On JOIN, signaling resolves nearest SFU pod via GeoDNS + RTT probe
- Anycast TURN ensures ICE candidates point at the closest relay
- Pod has soft cap of 5K sessions, hard cap of 7K; signaling rebalances under 80% utilisation

Cross-Region Cascading

When participants are spread across regions, attaching them all to a single SFU forces transatlantic uplinks. Instead, each region runs a local edge SFU and the SFUs **cascade** via a private backbone. Each publisher's stream traverses any inter-region link **at most once**, regardless of how many subscribers in that remote region.

Capacity Planning

- Per SFU pod: ~5K participants peak, ~15 Gbps egress
- Per region: 600–800 pods → ~3M participants
- 30 regions → ~100M global concurrent
- Recording composers are a separate pool: ~200 per region (fewer record than view)
- ASR pods autoscale per-language; English dominates ~60%

Why Spanner for Room State

- Strong consistency on roster join/leave (host kicks, lobby admit)
- Multi-region writes for follow-the-host migration when host changes region
- Interleaved tables: participants live under their room, single-row read for full roster
- Time-travel reads support post-mortem reconstruction of the join graph

13

Comparison: Meet vs Zoom vs Teams

Where the products differ

Aspect	Google Meet	Zoom	Microsoft Teams
Topology	SFU + simulcast (VP9/AV1)	Hybrid SFU (proprietary)	SFU (Skype stack roots)
Browser path	First-class WebRTC	WebRTC (recent); native preferred	WebRTC; native preferred
Max participants	1000 active video	1000 active; 10K webinar	1000 active; 20K live event
E2EE	Optional, <= 200 ppl	Optional, all sizes (since 2020)	Optional 1-on-1 only
Recording	GCS, server composed	Cloud or local; server composed	OneDrive/Stream; server composed
Captions	Streaming Conformer ASR; 30+ langs	On-device + cloud ASR	Azure Cognitive Services ASR
Codec preference	VP9-SVC then AV1	H.264 SVC then VP8	H.264 SVC

SDP vs JSEP

Spec	Role	Format
SDP (RFC 4566)	Describes a media session: codecs, IPs, ports, ICE	Plain-text, line-oriented (m=, a=, c=)
JSEP (RFC 8829)	JS API contract: createOffer / createAnswer / setLocalDescription	JS objects wrapping SDP strings
Trickle ICE (RFC 8838)	Stream ICE candidates as discovered	Per-candidate JSON over signaling
BUNDLE / RTCP-MUX	Multiplex audio+video on one transport	SDP attributes a=group:BUNDLE

14

Failure Modes & Recovery

What can go wrong and how we cope

Failure	Impact	Detection	Mitigation
SFU pod dies	All sessions on that pod drop	Health check + RTP heartbeat	Signaling triggers ICE restart on clients; reassigns to neighbor pod within 5 s
Region outage	All edge SFUs in region down	Anycast withdrawal, GCLB alarms	GeoDNS reroutes to next-nearest region; cascade links absorb traffic
TURN saturation	Symmetric-NAT clients can't connect	Allocation failure rate alarm	Autoscale TURN; downgrade non-essential clients to relay-via-SFU only
Bandwidth drop on receiver	Frozen video, dropped audio	TWCC reports, NACK storm	Layer downgrade L3 → L2 → L1; eventually audio-only
Encoder CPU pegged on sender	Sender drops simulcast layers	client stats getStats()	Disable L3 layer; reduce framerate to 15 fps
Recording composer crash	Recording gap	Heartbeat to orchestrator	Restart pod; resume from last fMP4 segment boundary
ASR backend slow	Captions lag	End-to-end caption latency p99	Drop to partial-only mode; queue NMT translations
Signaling WS drops	Roster/control lost; media still flows briefly	WS heartbeat	Client reconnects with session token; resumes without re-negotiating SRTP keys

Disaster Recovery

- **RTO:** < 30 s for SFU pod failover; < 5 min for region failover
- **RPO:** 0 for live media (calls just continue); < 10 s of recording lost on composer restart
- **Spanner:** multi-region synchronous; survives full region loss with no data loss
- **Recordings:** dual-region GCS bucket; checksums verified before original deletion

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Topology	SFU (mesh for 2 ppl)	Server cost vs uplink fairness. P2P preserves privacy + zero server but explodes $O(N^2)$; SFU adds a relay but linear uplink.
Adaptation	Simulcast 3 layers	3× sender CPU vs server transcoding cost. Simulcast wastes ~25% sender CPU; transcoding would cost 100× more in GPUs.
E2EE	Optional, off by default	Privacy vs feature set. E2EE breaks server-side recording, captions, noise cancellation, dial-in. Off by default keeps the suite usable.
Recording quality	1080p composed	Storage / compute vs fidelity. 4K would 4× storage with marginal viewer benefit; 720p saves 30% but looks dated for screen-share.
Region pinning	Pin to host region	Latency vs cost. Pinning to host minimizes inter-region cascade fees; worse for participants on the far side. Cascading limits damage.
Active-speaker rank	Audio energy + hysteresis	Simplicity vs accuracy. ML VAD is ~3% better but breaks E2EE and 10× more CPU.
Captions	Server-side streaming ASR	Latency + privacy vs accuracy + translation. On-device works for English-only; server-side needed for translation.
Signaling transport	WebSocket	Long-lived TCP vs ad-hoc HTTP. WS keeps state, handles trickle ICE; long-poll fallback for restrictive networks.

Real-time media is a popular interview because it forces you to talk about **networking**, **codecs**, **topology**, and **cost** simultaneously. Anchor the conversation around the topology choice — that decision drives almost every other answer.

45-Minute Interview Outline

1. **Intro (2 min):** clarify scale (100M concurrent), call shape (4 ppl × 30 min), browser-first, recording, captions, E2EE optional
2. **Capacity (5 min):** 10M calls/day, 40M sessions, ~833K avg / 100M peak; 20K SFUs across 30 regions
3. **Topology (8 min):** Mesh vs MCU vs SFU; defend SFU + simulcast; mention mesh-for-2
4. **Signaling vs media (5 min):** WebSocket SDP/ICE control plane; UDP/SRTP media plane; TURN for NAT
5. **Adaptive streaming (8 min):** simulcast L1/L2/L3, layer selection per receiver, TWCC + GCC, FEC under loss
6. **Active speaker (5 min):** RTP audio-level extension, EMA + hysteresis, top-N forwarding
7. **Recording + captions (5 min):** phantom participant for both; FFmpeg compose to GCS; ASR over data channel
8. **Failures + trade-offs (7 min):** SFU pod failover, region cascade, BW drop ladder; E2EE-vs-features call-out

Key Talking Points

- “SFU forwards, MCU mixes” — one sentence that locks in the topology choice
- “Simulcast lets each receiver pick its own layer” — explains adaptation
- “Active speaker is just RTP audio-level + hysteresis” — shows you know the protocol
- “Recording is a phantom participant” — elegant reuse of the same subscribe path
- “Cascade SFUs across regions, not single global SFU” — shows global awareness
- “Audio over video, always” — cite DSCP marking and FEC/RED
- “E2EE breaks recording and ASR” — shows you understand product trade-offs

Common Follow-ups & Answers

- **Q: Why not WebTransport / QUIC for media?** A: Coming, but RTP/SRTP over UDP has 20 yrs of NAT/middlebox compatibility and bespoke congestion control (GCC/TWCC). QUIC datagram is a likely future, currently used for some Meet experiments.

- **Q: How do you keep latency < 150 ms with the SFU in the path?** A: SFU adds ~5 ms forwarding; the bulk is encoder/decoder + jitter buffer. Edge SFUs sit in the same metro as users, and the only inter-region hop is a single cascade link.
- **Q: How does mute / camera off propagate?** A: Sender stops the encoder and announces state via signaling; SFU stops forwarding that SSRC and broadcasts a 'video paused' event. Receivers render the avatar tile.
- **Q: How do you handle 10K-person webinars?** A: Cascade through a tree of SFUs; only top-N speakers travel as full streams, the rest are audio+thumbnail; record once at root and stream HLS to view-only audience.
- **Q: ICE keeps failing for some users — what now?** A: Force TURN-relay (UDP 443, then TCP 443, then TLS) so the connection looks like HTTPS to corporate firewalls.
- **Q: What if Spanner hiccups?** A: Live calls don't depend on Spanner per-frame; only join/leave does. We can serve in-progress calls from cached SFU state for several minutes; new joins fail with retry.

KEY

Memorize These Numbers

100M concurrent peak · 10M calls/day · 20K SFU boxes · 30 regions · L1 150 kbps / L2 500 kbps / L3 1.5 Mbps · Top-9 active speakers · p50 mouth-to-ear < 150 ms · Recording 15 MB/min.